

GEM-garfield-simulation

インストールガイド

最終確認: 2026-09-26

目次

1	概要	1
1.1	インストール順序が重要な理由	1
1.2	動作確認済みのバージョンの組み合わせ	2
1.3	システムレベルのコンパイラ要件	2
2	ROOT	3
3	Gmsh	3
3.1	Gmsh CLI(任意)	3
3.2	Gmsh Python API(必須)	4
4	Elmer FEM	4
5	Garfield++	5
5.1	ROOT アップグレード後の再ビルド	5
5.2	事前に知っておくべき CMake の落とし穴	5
6	Python 環境	6
6.1	vtk のバージョン固定 (必須)	6
6.2	本プロジェクトが実際に動作確認しているバージョン	6
7	本プロジェクトの C++ マクロのビルド	7
8	パス設定がどこにあるか (シェル起動ファイルに依存しない)	7
8.1	任意: ネットワークファイルシステム上でのシェル起動高速化	7
9	パイプライン全体のエンドツーエンド動作確認	8
10	既知の互換性の落とし穴まとめ	8

1 概要

本書は、GEM-garfield-simulation が依存するソフトウェアスタック—— **Gmsh**(ジオメトリ・メッシュ生成)、**Elmer FEM** (静電場ソルブ)、**Garfield++/Magboltz**(microscopic な電子雪崩シミュレーション)、**ROOT**(I/O および、間接的に Garfield++ がビルドされるべき ABI を規定する)、そして解析・可視化に使う **Python** 環境——の、詳細でステップバイステップなインストールガイドである。

リポジトリルートの README.md は、既にセットアップ済みの環境に戻ってきたユーザーが最低限必要とする情報 (バージョン表とパス設定の場所) だけを載せている。本書はそれに対して、このスタックを初めてセットアップする、あるいは何かをアップグレードした後に一部を再構築するユーザー向けの、より詳しいリファレンスである。

1.1 インストール順序が重要な理由

このスタックの面倒ごとの大半を引き起こしている制約はただ一つ: **Garfield++** は、パイプラインの他の部分を使うのとまったく同じ **ROOT** インストール (同じバージョン、同じコンパイラ ABI) に対してビルドされていなければならない。ROOT の辞書生成 ABI はバージョン間で変わるため、ROOT 6.32 向けにリンクされた Garfield++ ビルドは、ROOT 6.40 の実行ファイルに対して以下のようなエラーでリンクに失敗する:

```
undefined reference to `ROOT::TGenericClassInfo::TGenericClassInfo(...)'  
undefined reference to `TGeoBBox::ComputeNormal(...)'
```

これを回避する方法は、一貫した 1 つの ROOT バージョンを維持し、そのバージョンが変わるたびに Garfield++ を再ビルドする以外にない。そのため、推奨されるインストール順序は以下の通り:

1. まず ROOT をインストールし、使い続けるバージョンを決める。
2. Gmsh・Elmer をインストールする (ROOT/Garfield++ とは独立)。
3. その ROOT インストールに対して Garfield++ をビルドする。
4. Python 環境をセットアップする (独立しているが、6 節に実際のパッケージ競合の落とし穴がある)。
5. 同じ ROOT + Garfield++ の組み合わせに対して、本リポジトリ自身の C++ マクロをビルドする。

1.2 動作確認済みのバージョンの組み合わせ

以下は、本プロジェクトが実際にビルド・動作確認 (2026-09-26 時点) している組み合わせである。いずれか 1 つのコンポーネントの後続のポイントリリースが他のコンポーネントと互換であることは保証されない (再検証が必要)。

ツール	バージョン
Gmsh (CLI)	4.13.1
Gmsh Python API (pip install gmsh)	4.15.2
Elmer FEM	v9.0
Garfield++	パッケージバージョン文字列 “0.3”(下記 ROOT に対してビルド)
ROOT	6.40.04
CMake	3.31.8
gcc	11.5.0 (Red Hat 11.5.0-14)

1.3 システムレベルのコンパイラ要件

以下のビルドを始める前に、システムに以下が揃っていることを確認する:

- C++17 対応の C++ コンパイラ (ここでは gcc/g++ 11.5.0。比較的新しい gcc/clang であれば何でもよい)、
- Fortran コンパイラ (gfortran)、
- CMake 3.16 以上。

Fortran コンパイラは見落としやすい: Garfield++ 自身の CMakeLists.txt は無条件に `project(... LANGUAGES Fortran CXX)` を宣言しており、これは Magboltz (Garfield++ が呼び出すガス輸送計算コード) が Fortran で書かれているためである——Fortran コンパイラが無いと、Garfield++ のビルドは cmake の段階で即座に失敗し、エラーメッセージに Fortran コンパイラのチェック名が現れる。

GSL (GNU Scientific Library) はここでは不要である——一部の古い Garfield++ のドキュメント/メモがこれを前提条件として挙げていることがあるが、これは Garfield++ 自身の CMakeLists.txt (`option(GARFIELD_WITH_GSL "Build Garfield++ with GSL (useless if C++>=17)" OFF)` —— C++17 より古い標準でビルドする場合のみ有効化・必須になる) を実際に確認して確定させた。本プロジェクトの Garfield++ ビルドは一貫して C++17 を使用しているため、GSL はインストールもリンクもされない。

2 ROOT

Garfield++ のビルドが ROOT に依存するため、まず ROOT をインストールする。自分のプラットフォーム向けに ROOT 自身のビルド手順 (https://root.cern/install/build_from_source/) に従うこと。ソースからビルドする場合の概略は以下の通り:

```
git clone --branch v6-40-04 --depth 1 https://github.com/root-project/root.git root_src
```

```
mkdir root_build && cd root_build
cmake -DCMAKE_INSTALL_PREFIX=$HOME/local/root/6.40.04 ../root_src
cmake --build . -j"$(nproc)"
cmake --install .
```

インストール後、使用前に ROOT の環境スクリプトを source する:

```
source $HOME/local/root/6.40.04/bin/thisroot.sh
root-config --version # 動作確認
```

本プロジェクトは `thisroot.sh` が (`.bashrc` などから) グローバルに source されていることに依存しない——各スクリプト・ビルド手順が必要なパスをその都度明示的に設定する (8 節参照)。これは、日常作業で使うシェルの起動が、毎回 ROOT 環境一式を読み込むことで遅くならないようにするための設計である。自分のワークフローに合わせて好きなようにセットアップしてよいが、以降のすべてのツールがどの ROOT インストールを指しているか、一貫させること。

3 Gmsh

必要なのは独立した 2 つのもの: Gmsh CLI(このプロジェクトは Gmsh をほぼ全面的に Python API 経由で操作するため、付随的にしか使わない) と、Gmsh Python モジュールである。

3.1 Gmsh CLI(任意)

<https://gmsh.info/#Download> からビルド済みバイナリをダウンロードして展開する。例:

```
mkdir -p ~/local
cd ~/local
wget https://gmsh.info/bin/Linux/gmsh-4.13.1-Linux64.tgz
tar xf gmsh-4.13.1-Linux64.tgz
~/local/gmsh-4.13.1-Linux64/bin/gmsh --version
```

3.2 Gmsh Python API(必須)

```
pip install gmsh
python3 -c "import gmsh; print(gmsh.__file__)"
```

pip パッケージ自体が同梱するバージョン (ここでは 4.15.2) は、上記のスタンドアロン CLI のバージョンと厳密に一致している必要はない——両者は独立したインストールであり、本プロジェクトは実行時に Python API しか実際には呼び出さない。

import の順序に関する落とし穴: 一部の conda/pip 環境では、同一の Python プロセス内で matplotlib より先に gmsh を import すると以下のエラーになる:

```
ImportError: /usr/lib64/libstdc++.so.6: version `CXXABI_1.3.15' not found
```

Gmsh のネイティブ拡張がシステムの古い `libstdc++.so.6` を先に読み込んでしまうことがあり、その SONAME がプロセス全体で一度読み込まれると、`matplotlib` のコンパイル済み拡張 (より新しい `libstdc++` ABI を要求する) は本来必要なバージョンを読み込めなくなる。修正は純粹に Python の `import` 順序レベルの話である: **両方を使うスクリプトでは、必ず `import gmsh` より先に `import matplotlib` (あるいは `matplotlib` を `import` する何らかのモジュール) を行うこと**。本プロジェクト自身のスクリプトは既にこの規約に従っている。両方に触れる新しいスクリプトを追加する場合もこれを維持すること。

4 Elmer FEM

ソースからビルドする (<http://www.elmerfem.org/blog/binaries/> に一部プラットフォーム向けのビルド済みバイナリもあるが、ソースからのビルドの方がクラスタ間の移植性が高い):

```
git clone https://github.com/ElmerCSC/elmerfem.git elmer_src
mkdir elmer_build && cd elmer_build
cmake -DCMAKE_INSTALL_PREFIX=$HOME/elmer \
      -DWITH_MPI=OFF \
      ../elmer_src
cmake --build . -j"$(nproc)"
cmake --install .
```

`-DWITH_MPI=OFF` は意図的な単純化である: 本プロジェクトのメッシュは 1 回の実行につき 1 つの線形システムとして解かれる (領域分割されない) ため、`ElmerSolver` 内での MPI 並列化は不要であり、MPI 無しビルドにすることで、クラスタ上の共有/中央集権的な Elmer インストールにありがちな共有ライブラリのバージョン不一致問題 (例: システムの `libmpi_usempif08.so` の不一致) のクラス全体を回避できる。自分の環境に既に動作する MPI 有効な Elmer インストールがあるなら、それを使ってもよい——ただし `ElmerSolver` が実際に動くことを確認すること (共有 HPC インストールの中には、CMake/リンクの段階は通過するのに、実際に呼び出した瞬間に共有ライブラリエラーで初めて壊れていることが判明するものもある)。

以下で動作確認する:

```
export ELMER_HOME=$HOME/elmer
export PATH="$ELMER_HOME/bin:$PATH"
export LD_LIBRARY_PATH="$ELMER_HOME/lib:${LD_LIBRARY_PATH:-}"
ElmerSolver -version
```

5 Garfield++

公式ソース (<https://gitlab.cern.ch/garfield/garfieldpp>) から、セクション 2 の ROOT インストールを明示的に指定してビルドする:

```
git clone https://gitlab.cern.ch/garfield/garfieldpp.git garfieldpp_src
mkdir -p garfieldpp_src/build && cd garfieldpp_src/build
```

```
export ROOTSYS=$HOME/local/root/6.40.04
export PATH="$ROOTSYS/bin:$PATH"
export LD_LIBRARY_PATH="$ROOTSYS/lib:$LD_LIBRARY_PATH"

cmake -DROOT_DIR="$ROOTSYS/cmake" \
      -DCMAKE_INSTALL_PREFIX=$HOME/local/garfield \
      ..
cmake --build . -j"$(nproc)"
cmake --install .
```

5.1 ROOT アップグレード後の再ビルド

ROOT をアップグレードした場合、Garfield++ は**必ず**新しいインストールに対して再ビルドしなければならない (上記と同じ cmake/build/install の手順を、ROOTSYS を新しい ROOT に向けて再実行する) —— 古い libGarfield.so を新しい ROOT に対してそのまま使い回すのは、まさにセクション 1.1 で述べた ABI 不一致そのものである。Garfield++ のインストールが他プロジェクトと共有されている場合は、その場で再ビルドする前に、依存している他の人に確認すること。

5.2 事前知っておくべき CMake の落とし穴

少なくとも 1 つの環境では、Garfield++ を利用するプロジェクト (本リポジトリ自身の macros/CMakeLists.txt を含む) の新規の cmake .. が、CMAKE_PREFIX_PATH を正しく設定していても、ROOT_DIR を古い/誤った ROOT インストールに静かに解決してしまうことがある。これは、Garfield 自身の GarfieldConfig.cmake が内部で find_dependency(ROOT 6.0 COMPONENTS Geom Gdml) を呼んでおり、環境から継承された CMAKE_PREFIX_PATH 次第でこの解決の競合に勝ってしまうことがあるためである。もし cmake .. の出力の Found Vdt: ... のような行が誤った ROOT を報告していたら、明示的に強制する:

```
cmake -DROOT_DIR=$HOME/local/root/6.40.04/cmake ..
```

あるいは、利用側プロジェクト自身の CMakeLists.txt の CMAKE_PREFIX_PATH 設定の直後に set(ROOT_DIR "... CACHE PATH "" FORCE) を追加する (本リポジトリの macros/CMakeLists.txt は既にこれを行っている —— そのファイル自身のコメント参照)。

6 Python 環境

conda・venv・素の pip install --user のいずれでもよく、本プロジェクトは特定の環境マネージャに依存しない。以下をインストールする:

```
pip install gmsh numpy matplotlib uproot pyvista plotly
pip install "vtk==9.3.1"
```

6.1 vtk のバージョン固定 (必須)

pyvista を単体でインストールすると、その時点の最新の vtkwheel が引き込まれる。2026-09 時点でそれは 9.7.x だが、これは pyvista の HTML 出力パスが使う trame_vtk と非互換であり、シーンを HTML 出力向けに直列化する際に以下で失敗する (visualization/plot_triple_gem.py で使用):

```
TypeError: unhashable type 'VTKAOSArray_vtkFloatArray'
```

(pyvista をインストールした後に) 明示的に vtk==9.3.1 へピン留めすることで解決する。将来 trame_vtk のリリースがこの非互換性を修正した場合はこのピン留めを緩めてよいかもしれないが、それは前提とせず、検証した上での意図的な判断として扱うこと。

6.2 本プロジェクトが実際に動作確認しているバージョン

パッケージ	バージョン
numpy	2.4.4
matplotlib	3.10.9
uproot	5.7.4
pyvista	0.49.0
plotly	7.1.0
vtk	9.3.1(上記の通り固定)
gmsh (Python)	4.15.2

補足: 本プロジェクトの Python コードが ROOT ファイルを読むのに使うのは PyROOT ではなく uproot である。PyROOT は意図的に使用していないため、Python 環境自体のバージョンを ROOT 側の Python バインディングと一致させる必要は一切ない——追跡すべきバージョン制約が 1 つ減っている。

7 本プロジェクトの C++ マクロのビルド

ROOT・Garfield++・Elmer がインストールされ、互いに整合していれば:

```
cd GEM-garfield-simulation/macros
mkdir -p build && cd build
cmake ..
cmake --build . -j"${nproc}"
```

macros/CMakeLists.txt は、デフォルトで CMAKE_PREFIX_PATH を \$HOME/local/garfield と \$HOME/local/root/6.40.04 に向けている——自分の環境のインストール場所が本プロジェクトの開発機と異なる場合は、同ファイル冒頭のこの 2 つのパスを編集すること。

cmake .. が誤った ROOT を報告する場合は (6 節より前のセクション 3.3 参照)、そこに示した通

り `-DROOT_DIR=...` を明示的に渡す。

8 パス設定がどこにあるか (シェル起動ファイルに依存しない)

本プロジェクトは、`ROOTSYS`・`ELMER_HOME`・`GARFIELD_HOME` が `.bashrc`(など) からグローバルに `export` されていることに意図的に依存しない。各エントリポイントが必要なものをその都度明示的に設定する:

- `elmer/run_field_solve.sh` は、`ElmerGrid/ElmerSolver` を呼び出す前に自分自身で `ELMER_HOME/PATH/LD_LIBRARY_PATH` を `export` する。
- `macros/CMakeLists.txt` は、`configure` 時に自分自身で `CMAKE_PREFIX_PATH`(必要なら `ROOT_DIR` も) を設定する。
- Python スクリプトは `uproot`(純粋な Python) 経由で `ROOT` ファイルを読むため、`ROOT` 環境を一切必要としない。

上記のデフォルトのパスと異なる自分自身のパスをセットアップした場合、編集が必要なのは上の 2 ファイルだけである——他のどこかに隠れた第三の重複箇所は存在しない。

8.1 任意: ネットワークファイルシステム上でのシェル起動高速化

ホームディレクトリがネットワークファイルシステム (GPFS/NFS など) 上にあり、大きな `conda` 環境や `ROOT` インストールのせいで、セッションの最初の `python/root` 呼び出しが遅い (多数の小さいファイルへのネットワーク越し `stat/open` の積み重ねで数十秒かかる) 場合、その環境を 1 つの `squashfs` イメージにまとめて、ノードローカル/`/tmp` 配下に読み取り専用でマウントするのが有効な対策の一つである。本プロジェクトの開発機自体がまさにこの方式を使っている (そのマシン固有の実装詳細は、本リポジトリの外にある `~/local/envfs_README.md` 参照) —— これは純粋な性能最適化であり、完全に任意であり、本書の他の内容とは独立している: バッチ (LSF) ジョブは、実体としての (マウントされていない) パスを常に見るため、これの影響を一切受けない。

9 パイプライン全体のエンドツーエンド動作確認

上記すべてをインストールした後、`triple_gem_field` モデルを使って、ジオメトリ生成から Python 解析まで、パイプライン全体が実際に動くことを確認する (このコマンド列は `docs/reference.md` §2 で動作確認済みのものそのもの):

```
cd GEM-garfield-simulation

cd geometry && python3 build_triple_gem_field_mesh.py && cd ..
cd elmer && bash run_field_solve.sh triple_gem_field && cd ..

cd macros/build
./export_avalanche_trajectories ../../results/mesh/triple_gem_field \
  ../../resources/ar_ch4_90_10.gas 5 -0.2029 0.8405 0.4235 0.021 \
```

```
0.03637306695894642 0.1 0.0005 ../../results/root
cd ../../

python3 visualization/plot_triple_gem.py triple_gem_field
python3 geometry/analyze_plane_crossings.py \
  results/root/triple_gem_field_avalanche.root
```

正 常 に 完 了 す る と、 `results/root/triple_gem_field_avalanche.root` と `results/html/triple_gem_field_overview.html` が生成され、最後のコマンドが Python の traceback なしに `funnel/summary` を出力する。各パイプライン段の CLI 引数の意味 (別の `mesh/model` に対してそれらをどう計算するかを含む) や、7×7 production configuration のフル実行方法は `docs/reference.md` を参照。

10 既知の互換性の落とし穴まとめ

- **Fortran コンパイラの不足:** `gfortran` が無いと `Garfield++` のビルドは `cmake` の段階で即座に失敗する (`Magboltz` の実体が Fortran のため)。GSL は、古いドキュメントの記載に反して、C++17 ビルドでは不要 (セクション 1.3)。
- **Garfield++ / ROOT の ABI:** 使う ROOT と厳密に同じバージョンに対してビルドされていなければならない。ROOT をアップグレードしたら必ず `Garfield++` を再ビルドする (セクション 5.1)。
- **CMake が誤った ROOT_DIR を解決する:** 新規の `cmake ..` が誤ったものを選んだら明示的に強制する (セクション 5.2)。
- **gmsh と matplotlib の import 順序:** 両方に触れる同一プロセスでは、必ず `matplotlib` に触れるモジュールを `gmsh` より先に `import` する (セクション 3、`Gmsh`)。
- **vtk のバージョン:** 9.3.1 に固定すること——最新 wheel は `pyvista` の HTML 出力を壊す (セクション 6.1)。
- **共有/HPC インストール上の MPI 有効 Elmer:** ビルドが通ることだけでなく、`ElmerSolver` が実際に実行時に動くことを確認する——壊れた共有 MPI ランタイムは `CMake/リンク` の段階を通過してしまい、実際に呼び出して初めて共有ライブラリエラーで失敗することがある (セクション 4)。

パイプライン自体を組み立てて動かす過程で踏んだ落とし穴 (基盤ツールのインストールそのものではなく) については、本リポジトリの `docs/pipeline_gotchass.md` を参照。